

Conceptos Básicos de Inteligencia Artificial y Aprendizaje Automático para la Investigación

Clase 3: Aprendizaje Automático No-Supervisado

Dr. Juan Claudio Toledo Roy
Instituto de Ciencias Nucleares UNAM

Contenido de la clase

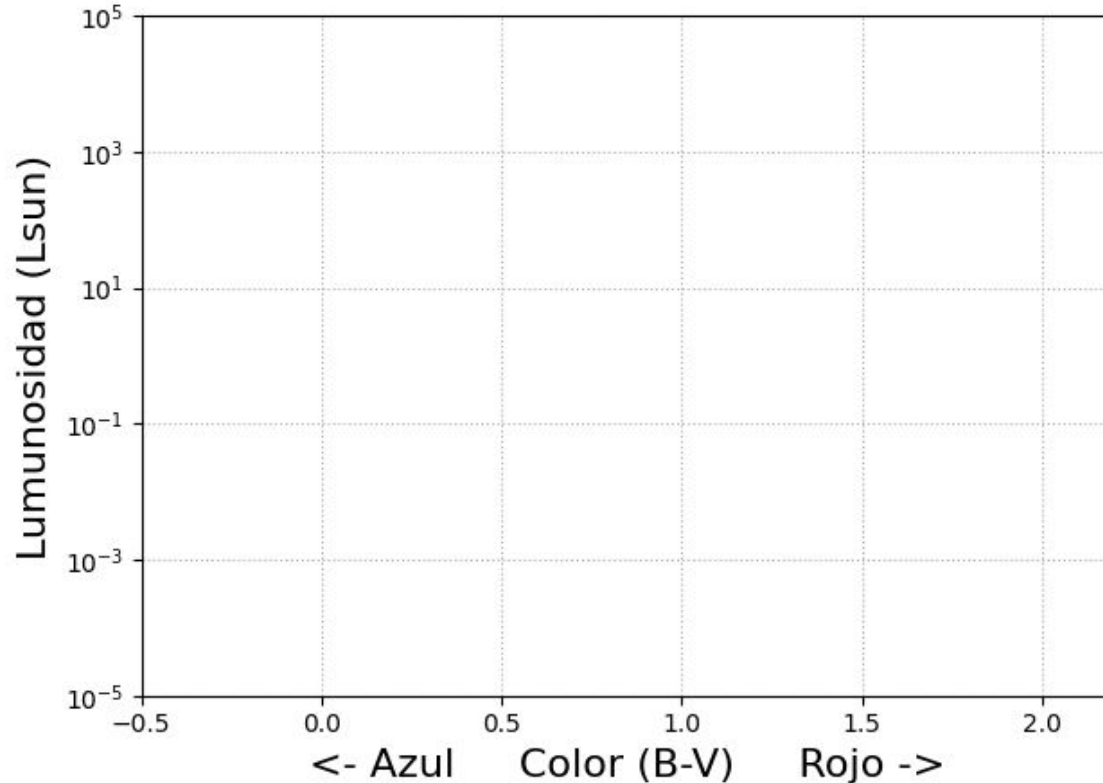
- ¿Qué es el Aprendizaje Automático No-Supervisado?
- Clustering
 - K-means, Clustering Jerárquico, DBSCAN
- Reducción de dimensiones
 - PCA, tSNE, UMAP
- Detección de anomalías
 - Isolation Forest

Ejercicios: https://github.com/meithan/Curso_IA_Investigacion

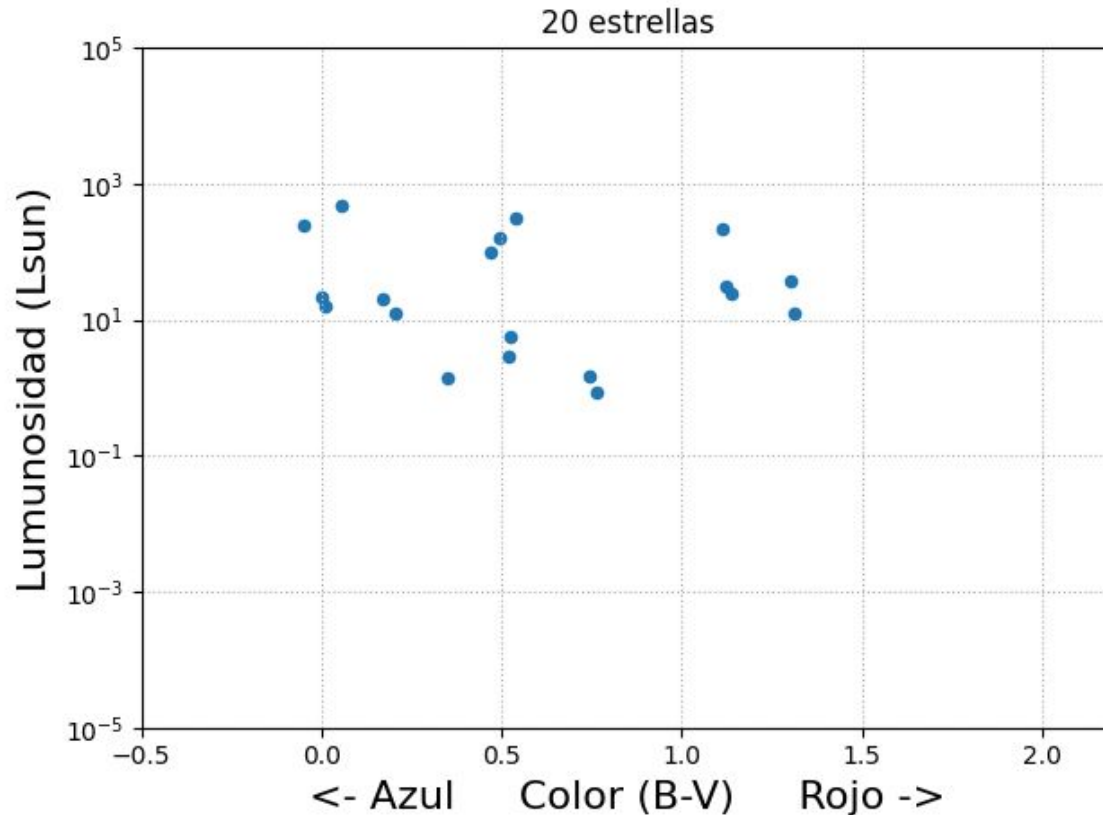
¿Qué es el Aprendizaje Automático No-Supervisado?

- No hay **clases** o **etiquetas** asociadas a los datos (como en aprendizaje supervisado)
- No nos interesa *predecir* algo, sino **descubrir** estructura
- Preguntas clave que nos interesan:
 - ¿Hay **agrupación** (¿clases?) en los datos?
 - ¿Podemos **reducir** las **dimensiones** sin perder mucho?
 - ¿Existen **anomalías**? (¿bajo qué criterio?)
- Buenas herramientas para fase exploratoria (EDA)
- Sobre todo útil para datos altamente multi-dimensionales.

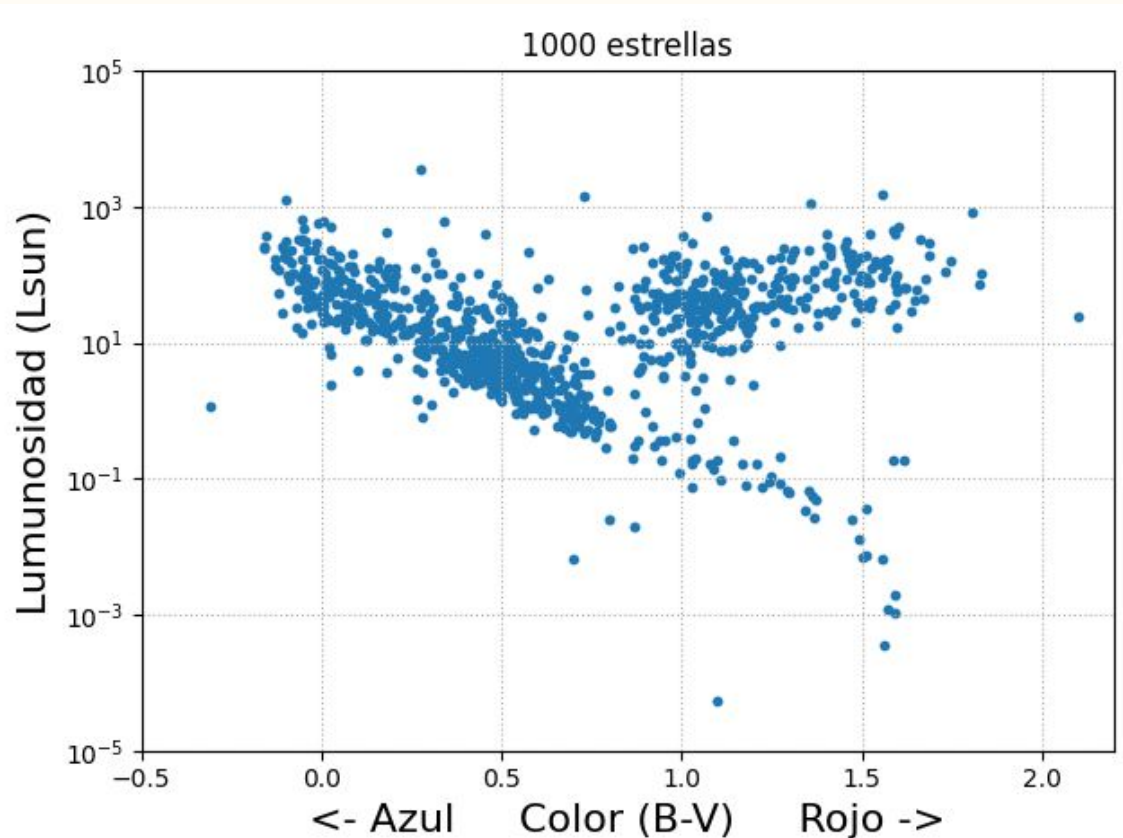
¿Qué es el Aprendizaje Automático No-Supervisado?



¿Qué es el Aprendizaje Automático No-Supervisado?



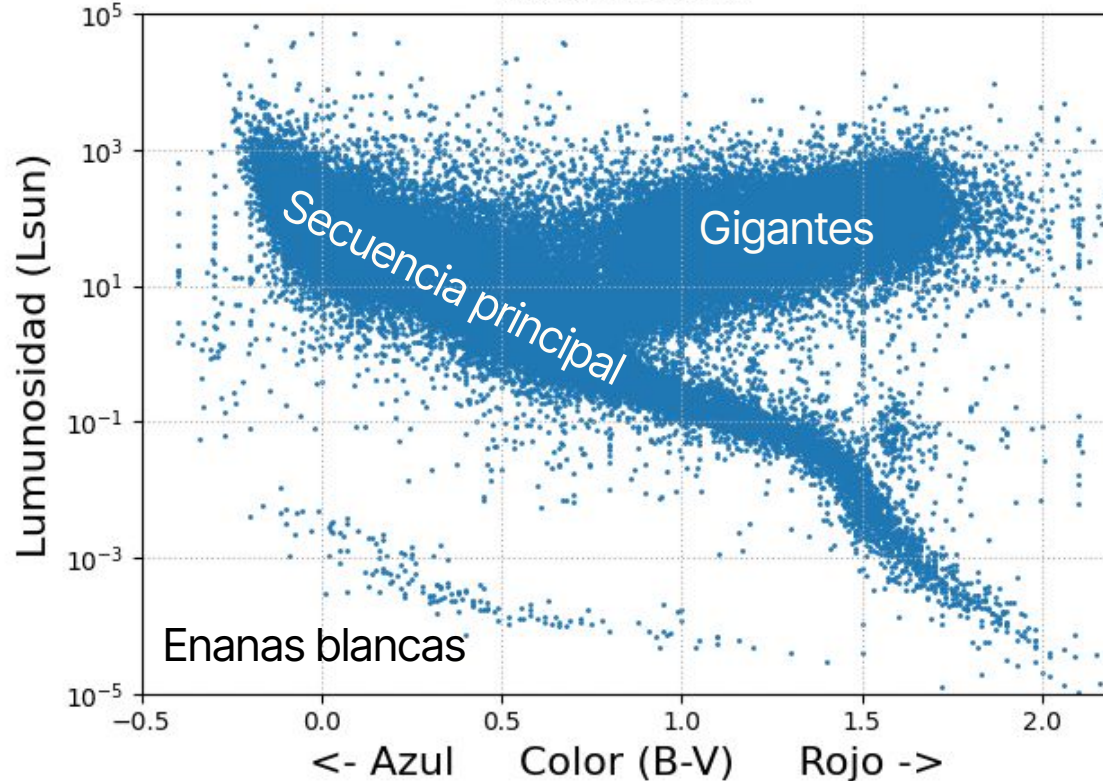
¿Qué es el Aprendizaje Automático No-Supervisado?



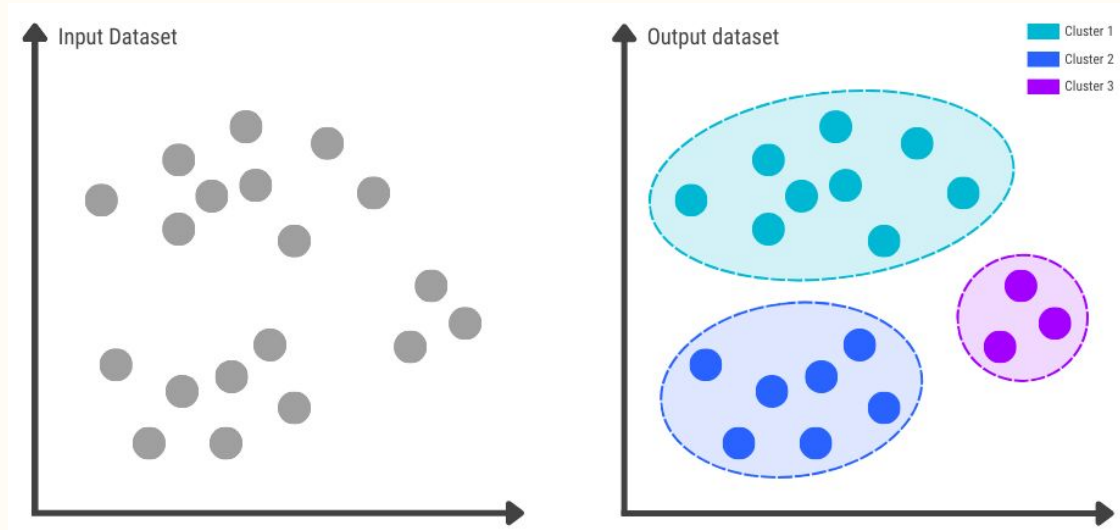
¿Qué es el Aprendizaje Automático No-Supervisado?

DIAGRAMA HERTZSPRUNG-RUSSELL

109401 estrellas



1. Clustering (agrupamiento automático)



- ¿Existen grupos/clases naturales en los datos? ¿Cuántos?
- ¿Existen unos pocos “prototipos” de los datos?
- Objetivo: Agrupar (automáticamente) datos para que puntos similares pertenezcan al mismo grupo y puntos disimilares a diferentes

1. Clustering (agrupamiento automático)

- “Similaridad” puede no ser trivial de definir, depende de datos:
 - Con base en características de los datos
 - Con base en alguna métrica de distancia (no necesariamente espacial)
- Además, hay varias estrategias (algoritmos) para crear las agrupaciones
- Usos:
 - EDA
 - Descubrimiento científico (poblaciones, fases, estados)
 - Pre-procesamiento para visualización o clasificación

1. Clustering: k-means

- Objetivo: **minimizar varianza dentro** de cada grupo
- Basado en una **métrica de distancia**
- Debemos especificar nosotros el **número de grupos k**
- Suposiciones:
 - Cada grupo es ~esférico (bajo la métrica definida)
 - Las variables son numéricas y comparables entre sí
 - Hay una noción de distancia (métrica)

- Matemática básica:

- A cada grupo (*cluster*) se le calcula su **centroide**:

$$\mu_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x}$$


La media
del cluster

- Escoger partición que minimiza
suma de varianzas *within-cluster*:

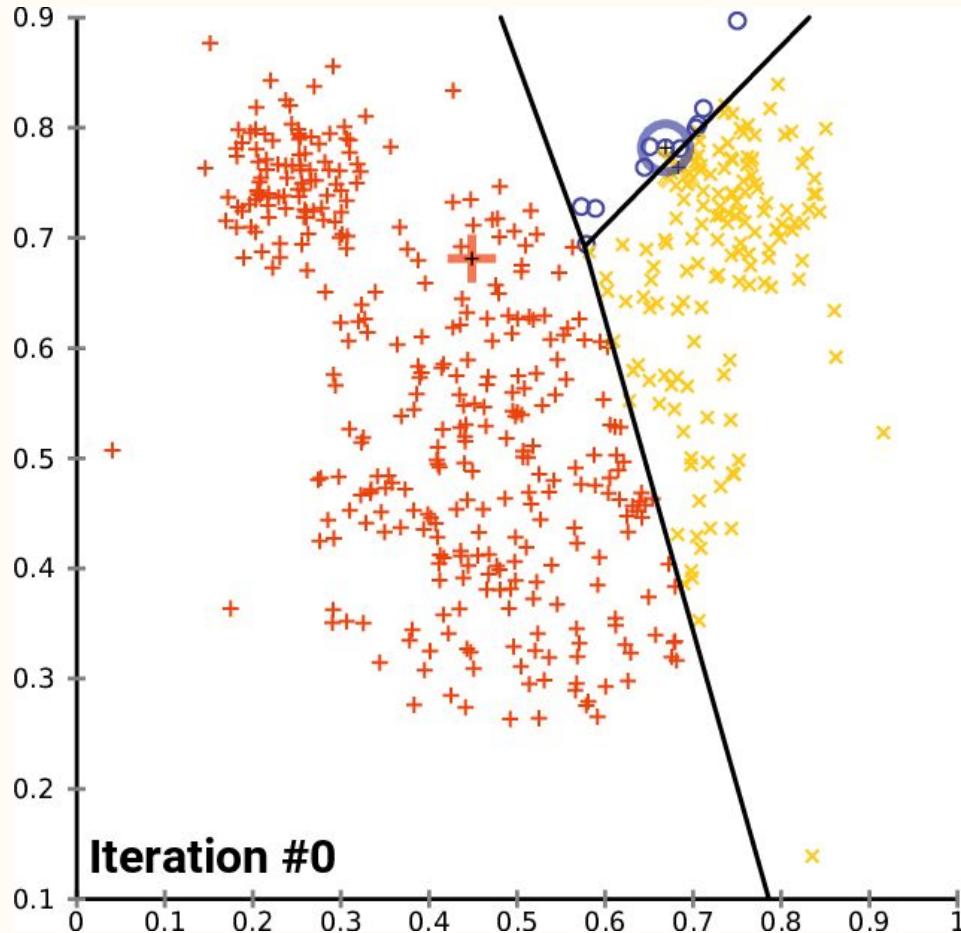
$$\sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2$$

(WCSS)

1. Clustering: k-means

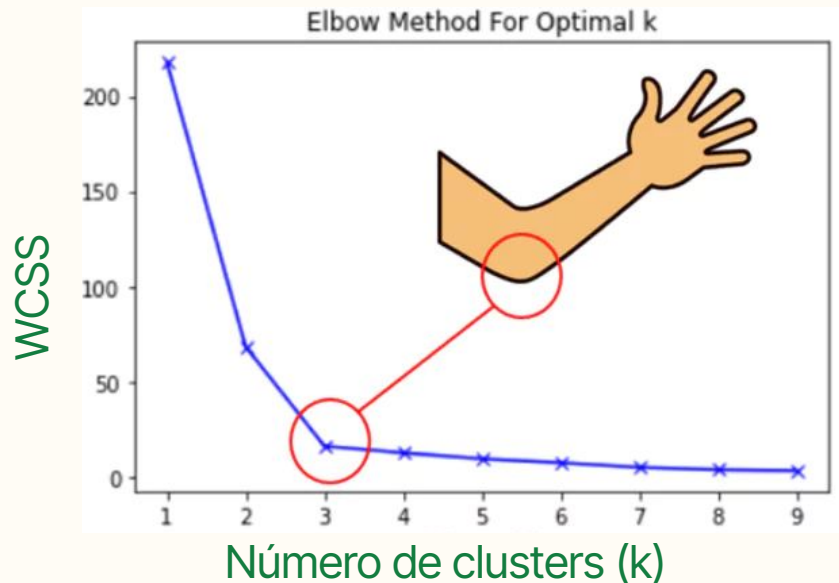
Algoritmo (básico):

- Partir de k centroides pre-escogidos (e.g. al azar)
- Iterativamente:
 - Para todos los puntos, determinar a cuál centroide está más cerca cada punto
 - Re-calcular el centroide de cada cluster
- Repetir hasta que ya no cambia la asignación puntos $\leftrightarrow$ clusters

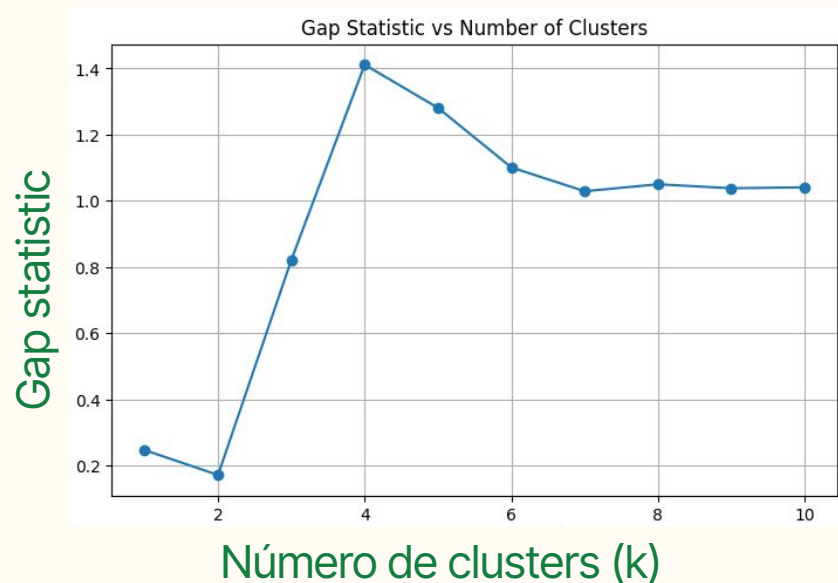


1. Clustering: k-means

- Muchas veces no sabemos a priori **cuántos** clusters hay (¡a veces justo eso queremos determinar!)
- Elbow method

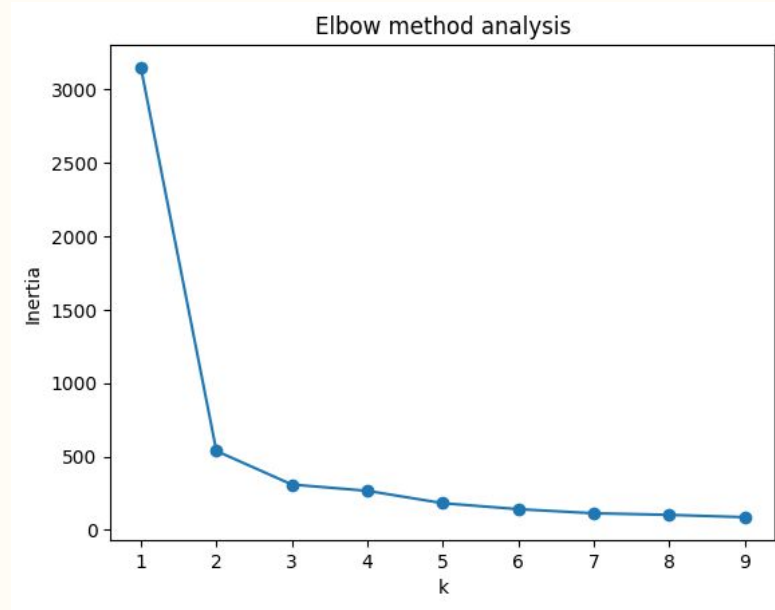
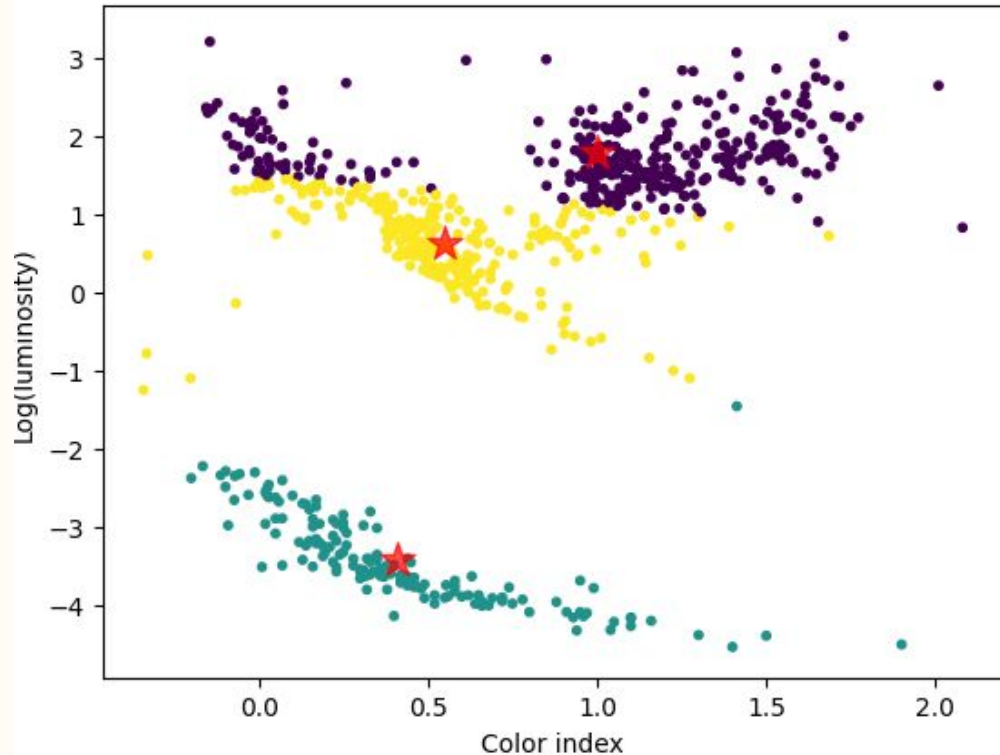


- Gap statistic



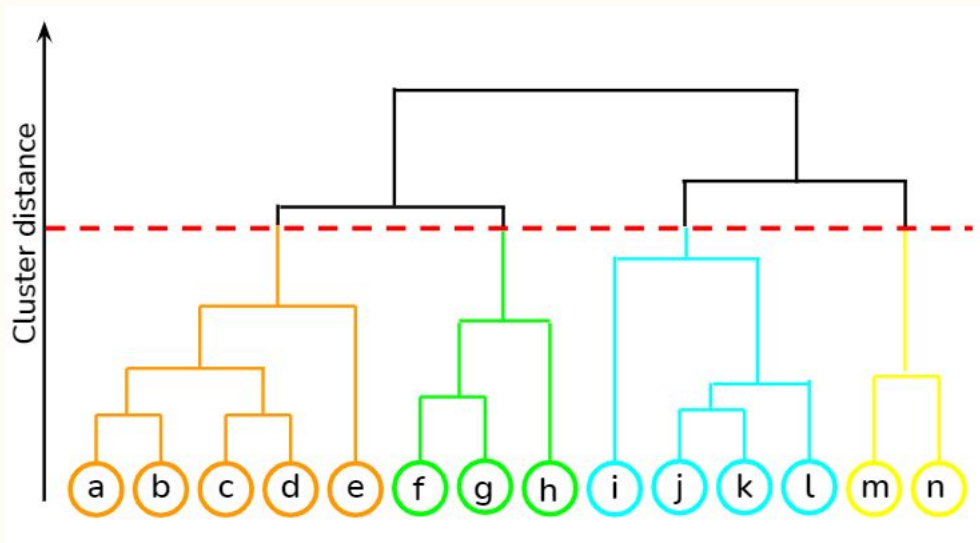
1. Clustering: k-means

Ejemplo: agrupar estrellas según color y luminosidad: [kmeans.ipynb](#)



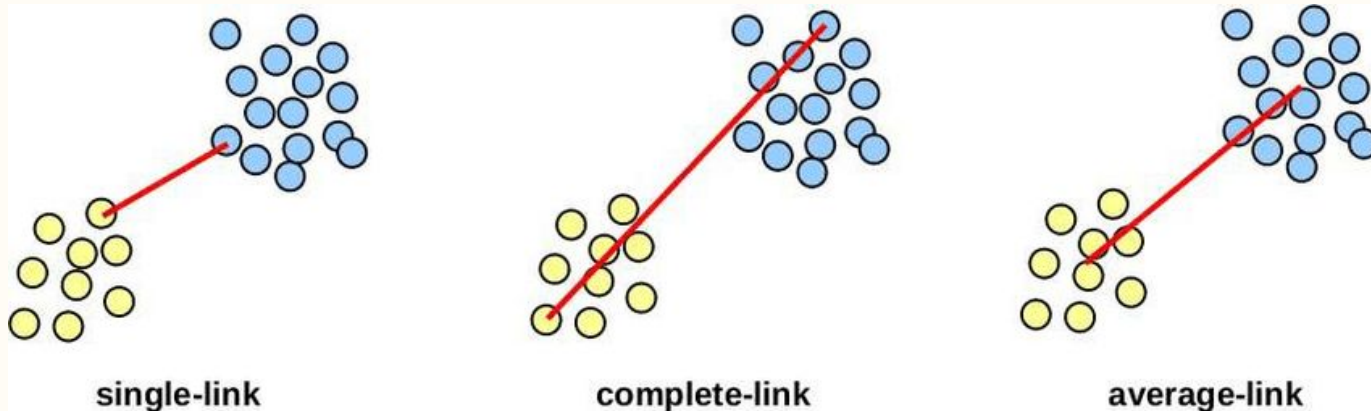
1. Clustering: Jerárquico (hierarchical clustering)

- Construir una **jerarquía** de diferentes niveles de agrupación
- El resultado es un **árbol** de jerarquía, en muchas escalas
- Podemos **extraer clusterings** a todos los niveles de **granularidad** (no escogemos el *número* de clusters, sino una *escala*)
- Se necesita una **métrica de distancia** (igual que en k-means)



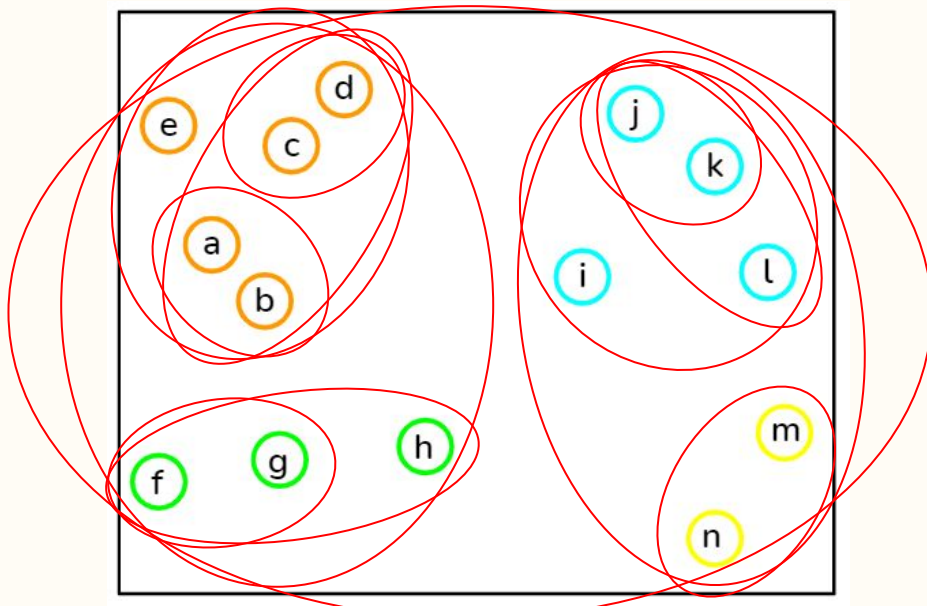
1. Clustering: Jerárquico (hierarchical clustering)

- *Linkage*: criterio a especificar para la distancia **entre dos clusters**
 - Distancia *mínima* entre parejas (single linkage)
 - Distancia *máxima* entre parejas de uno y otro (complete linkage)
 - Distancia *promedio* entre todas las parejas (average linkage)
 - Ward: incremento de la varianza al fusionar dos clusters



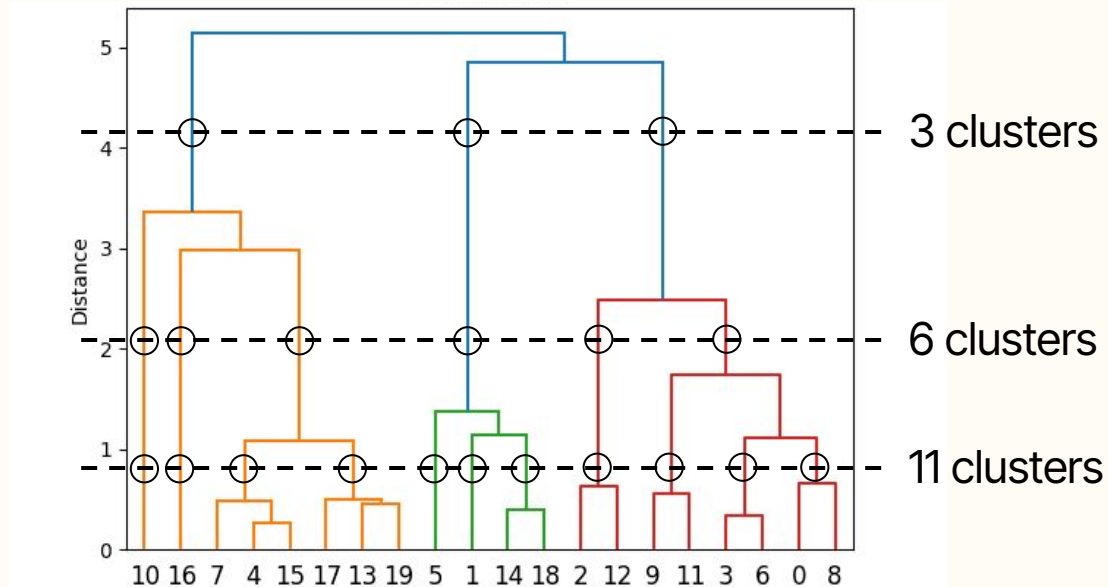
1. Clustering: Jerárquico (hierarchical clustering)

- Algoritmo básico (aglomerativo; también hay divisivo):
 - Cada punto empieza como su propio cluster (de un elemento)
 - En cada paso, fusionamos los dos clusters más cercanos
 - Terminamos cuando queda un solo cluster



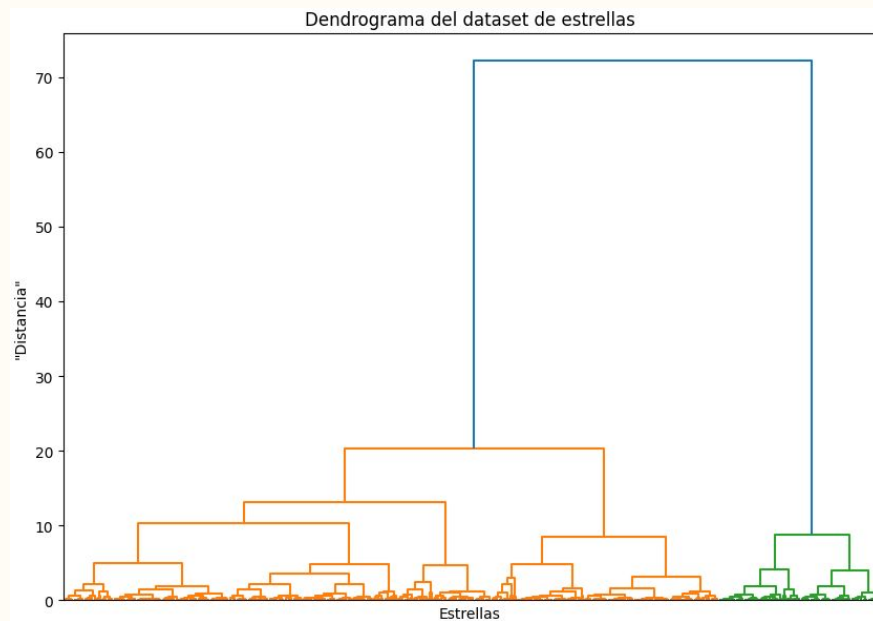
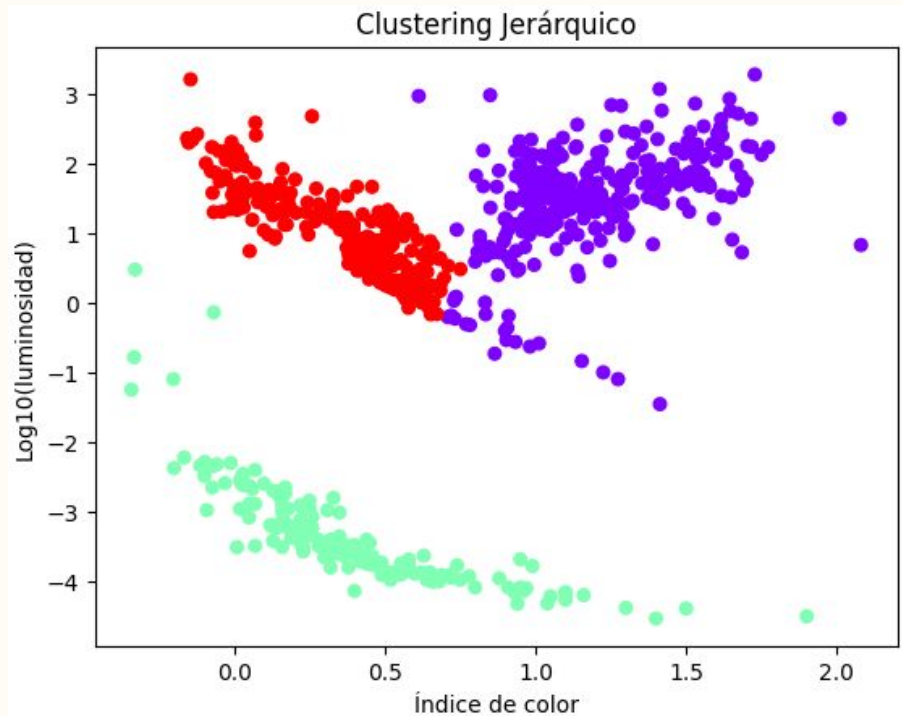
1. Clustering: Jerárquico (hierarchical clustering)

- El **dendograma** representa los clusters en orden jerárquico
- El eje y es la **distancia**, y es continua
- Podemos **cortar** el dendograma, obteniendo más o menos clusters



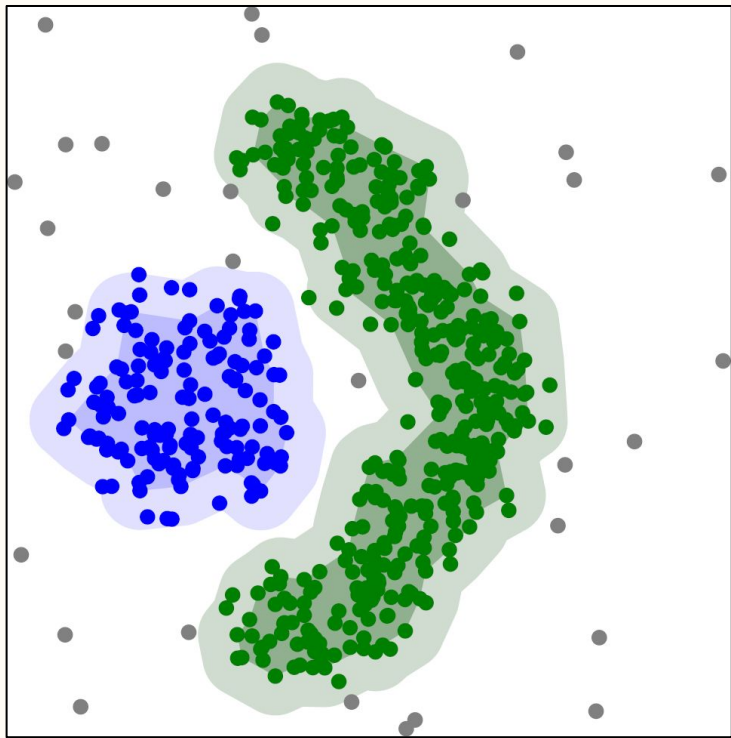
1. Clustering: Jerárquico (hierarchical clustering)

Ejemplo: [hierarchical.ipynb](#)



1. Clustering: DBSCAN

Density-Based Spatial Clustering of
Applications with Noise



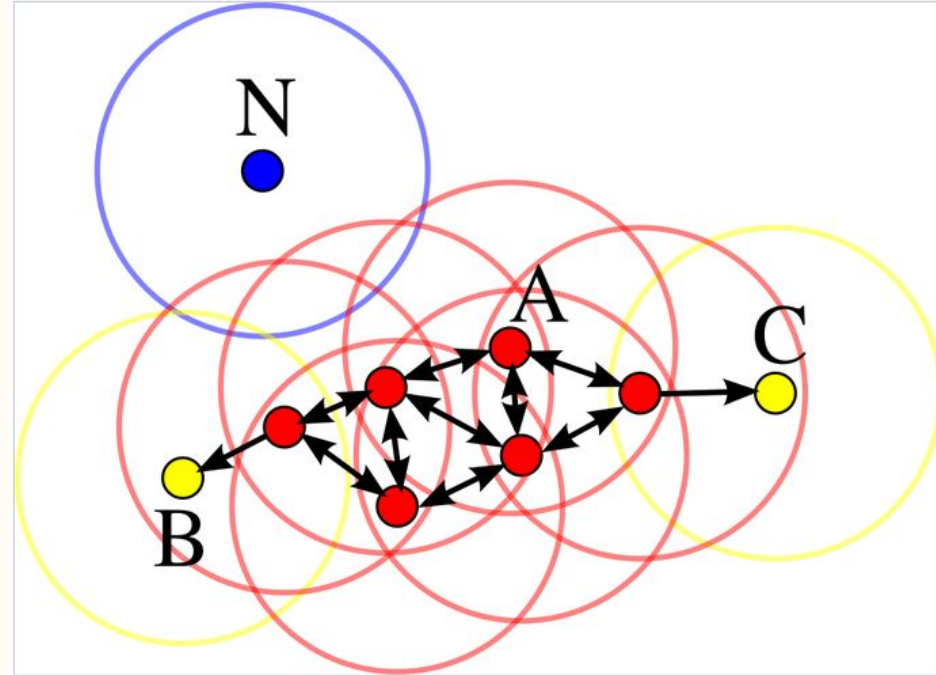
- Objetivo: agrupar basado en la **densidad local**, no distancia
- Clusters son **regiones densas** separadas por brechas de baja densidad
- Datos entre clusters son "ruido" (identifica **outliers**)
- Genera clusters con **formas arbitrarias** (no tienen que ser ~esféricos; es no-lineal)
- No necesitamos dar número de clusters
- Pero hay que dar:
 - ϵ : radio de vecindad de cada punto
 - min_pts: mínimo número de puntos para formar una vecindad

1. Clustering: DBSCAN

Density-Based Spatial Clustering of Applications with Noise

Idea general del algoritmo:

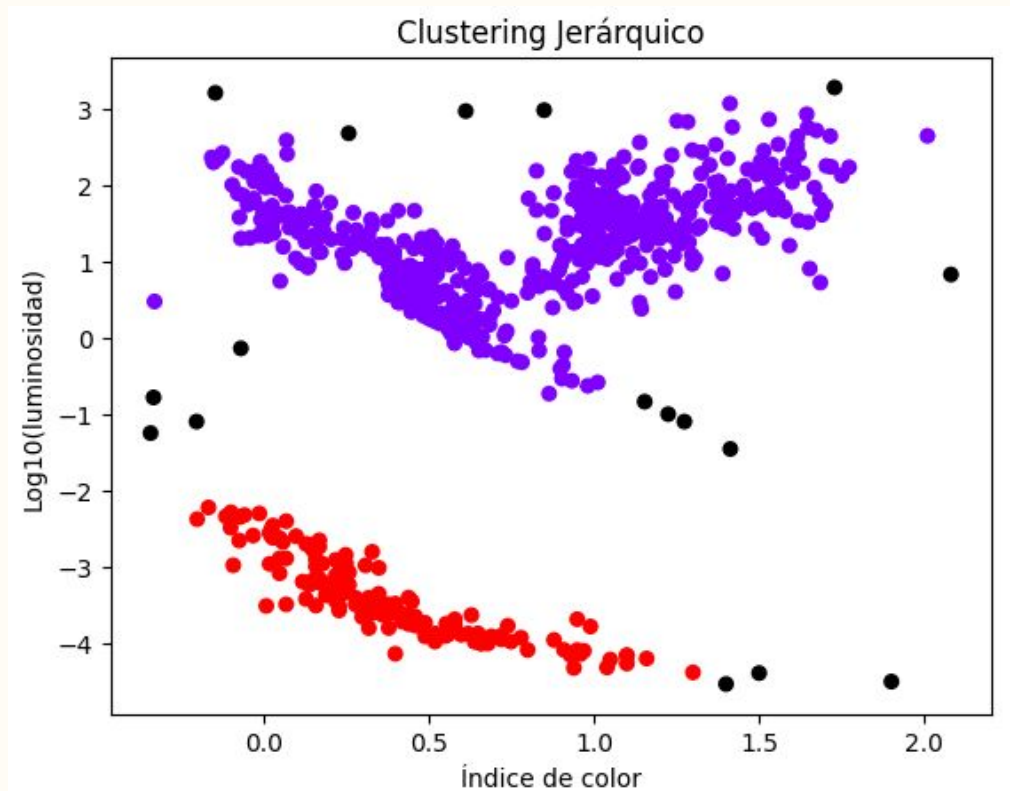
- Identificar todos los **core points**: aquellos que tienen al menos num_pts puntos en su epsilon-vecindad
- Tomar un core point, añadir vecinos a su cluster, y si vecino es otro core point, añadir los vecinos de ese
- **Border** points son aquellos en la periferia (dentro cluster pero no core points)
- **Noise** points lo que sobra



1. Clustering: DBSCAN

Density-Based Spatial Clustering of
Applications with Noise

Ejemplo: [dbscan.ipynb](#)

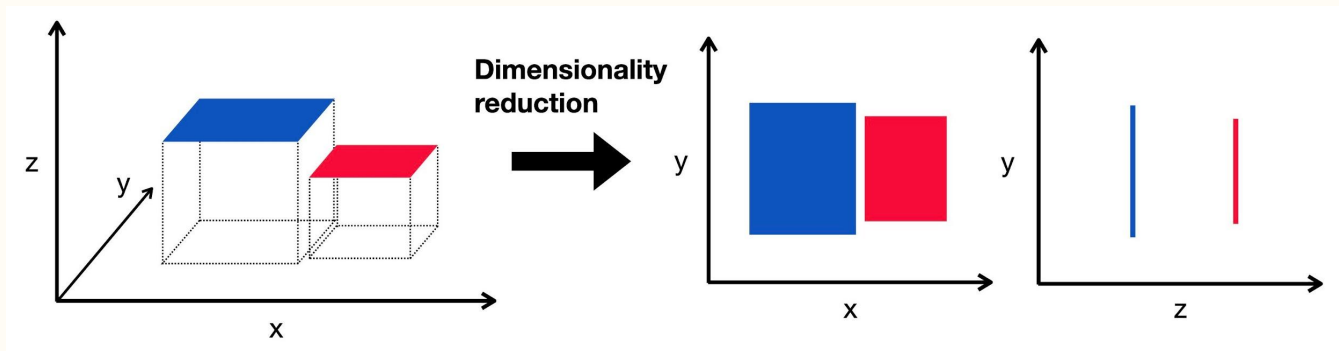


1. Clustering: Resumen

Método	Idea central	Fortalezas	Limitaciones	Usos sugeridos
k-means	Minimizar distancia a k centroides	• Simple y rápido de calcular • Escala a datasets grandes • Fácil de interpretar	• Hay que pre-definir k • Supone clusters esféricos • Sensible a escalamiento de variables y a outliers	• Datasets grandes • Clusters compactos y bien separados • Útil cuando tiempo de cómputo limitado
Jerárquico	Construir jerarquía anidada de clusters	• No hay que pre-definir k • Dendograma muy interpretable • Multi-escala	• Más caro computacionalmente • Sensible a ruido y a elección de métrica • Depende un poco de cómo inicia el algoritmo	• Análisis exploratorio • Datasets chicos a medianos • Cuando estructura multi-escala es deseable
DBSCAN	Clusters son regiones densas del espacio	• Clusters con cualquier forma • Identifica outliers • No hay que escoger k	• Problemas si densidad varía mucho • Requiere tuning de parámetros • Caro en muchas dims.	• Datos espaciales • Datos con mucho ruido / outliers • Clusters no-esféricos

2. Reducción de dimensiones

- Representar los datos **con menos variables** pero preservando su **estructura fundamental**



- ¿Por qué queríamos hacer eso?
Los datos con **muchas dimensiones** (variables) son difíciles de:
 - Visualizar (se puede 1D, 2D, maso 3D ... ¿pero 100D?)
 - Interpretar (¿qué hago con tantas variables?)
 - Analizar (*curse of dimensionality*)

2. Reducción de dimensiones

Lo que hace:

- Encontrar nuevas variables (combinando las existentes)
- Seleccionar las más importantes (varianza, información, localidad)

Permite:

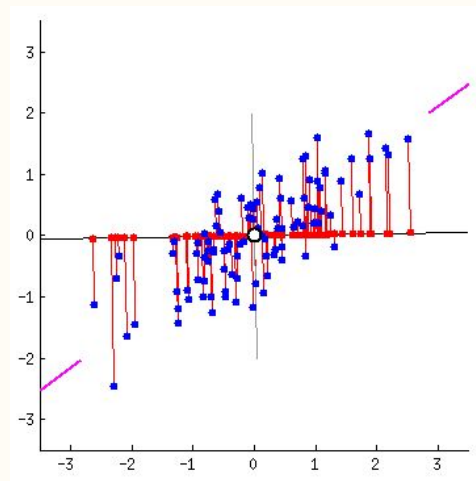
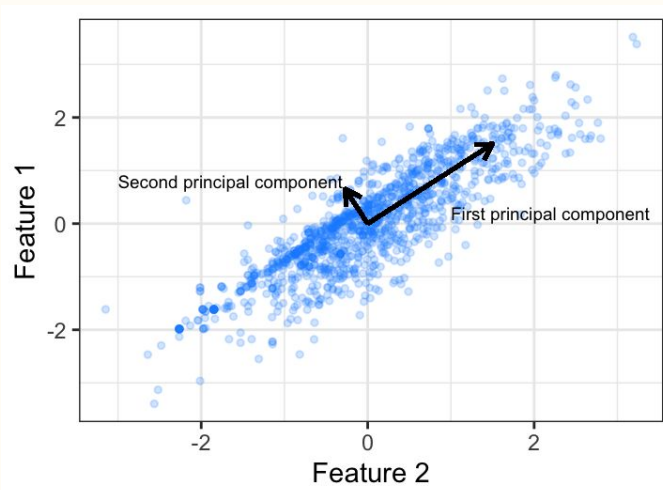
- Visualización efectiva
- Compresión y remoción de ruido
- Pre-procesamiento para clustering o clasificación

Dos grandes familias:

- Lineales: preservan varianza global (e.g. PCA)
- No-lineales: preservan estructura local (e.g. t-SNE, UMAP)

2. Reducción de dimensiones: PCA

- **Principal Component Analysis:**
Una técnica **lineal** de reducción de dimensiones que encuentra las direcciones (en N dims) de **máxima varianza**.
- Idea central: **rotar los ejes** del espacio de los datos para encontrar **nuevos ejes** que **amplifican la varianza** (al proyectar los datos)



2. Reducción de dimensiones: PCA

- Las **componentes principales** (PCs) encontradas son:
 - Combinaciones lineales ortogonales de las variables originales
 - Están ordenadas por cuánta varianza explicada contienen
 - Podemos quedarnos con las primeras solamente y tener mucha o casi toda la varianza (y entonces estructura) de los datos
 - Lo que es ruido se queda en las componentes poco importantes
- Cómo funciona (muy básico):
 - Centrar los datos (restar media o z-score)
 - Calcular matriz de covarianza de los datos
 - Encontrar eigenvalores y eigenvectores
 - Proyectar datos sobre cada componente

¡Es pura álgebra lineal!

$$z = \frac{x_i - \mu}{\sigma} \quad \mathbf{X} \rightarrow \mathbf{B}$$

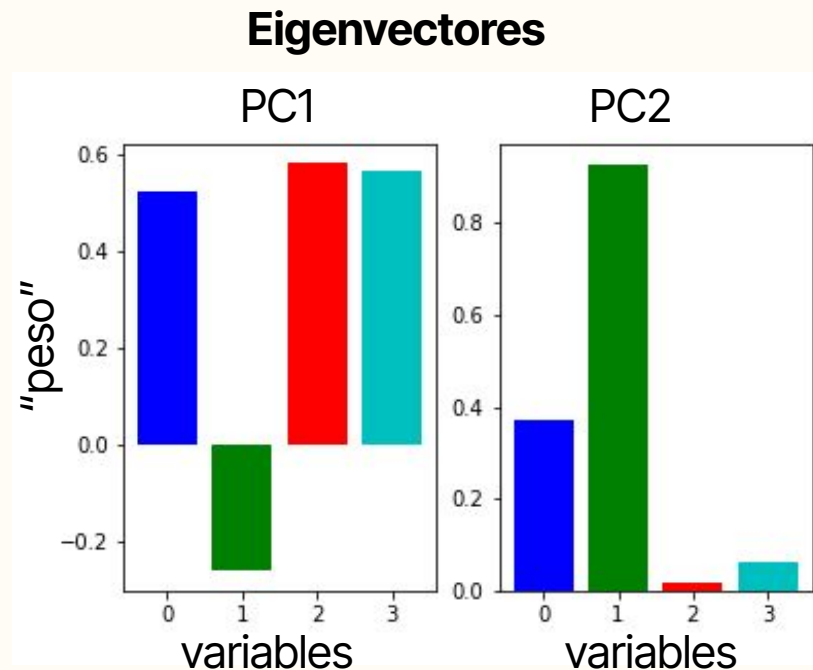
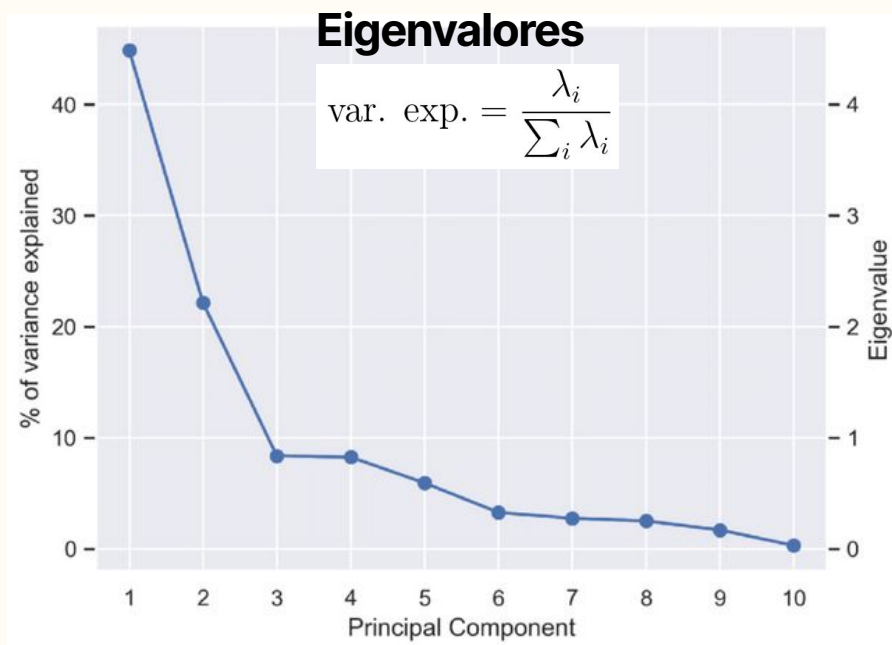
$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^T \mathbf{B}$$

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v} \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$$

$$\text{Data}_{new} = \mathbf{B} \cdot \mathbf{W}$$

2. Reducción de dimensiones: PCA

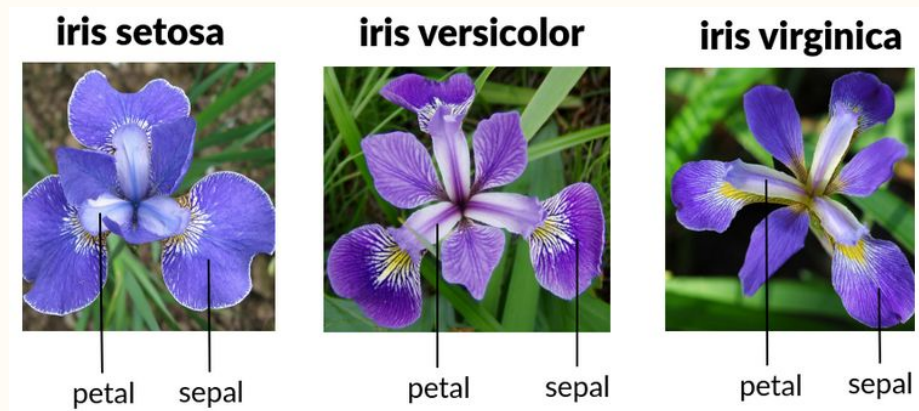
- **Eigenvalores:** dicen la **varianza explicada** de cada componente
- **Eigenvectores:** "pesos" para obtener cada PC a partir de vars. origs.



2. Reducción de dimensiones: PCA

Ejemplo: [pca.ipynb](#)

El famoso “iris dataset”



4 variables:

- Longitud pétalo
- Ancho pétalo
- Longitud sépalo
- Ancho sépalo

3 tipos de flores iris:

- Setosa, versicolor, virginica

150 ejemplos, 50 de cada tipo

2. Reducción de dimensiones: t-SNE

t-distributed **S**tochastic **N**eighbor **E**mbedding

SNE: Sam Roweis y Geoffrey Hinton

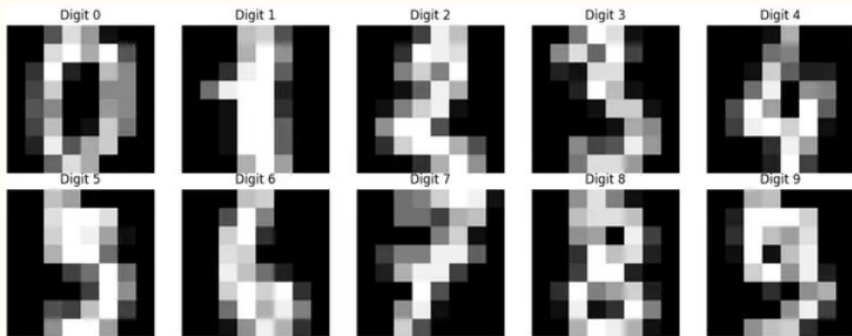
- Una reducción de dimensiones **no-lineal** diseñada para **visualizar** datos complejos de **muchas dimensiones** en 2D o 3D
 - Ideas centrales:
 - Puntos cercanos en **alta D** deben permanecer cercanos en **baja D**
 - Preserva **vecindarios locales**, no estructura global
 - Usa distribuciones de probabilidad de similitud entre variables, no distancias directamente
 - Sirve para:
 - Revelar clusters y sub-estructura
 - Explorar datos de estructura compleja y no-lineal
 - Visualizar embeddings (conversiones datos a números)

2. Reducción de dimensiones: t-SNE

Ejemplo: [tsne.ipynb](#)

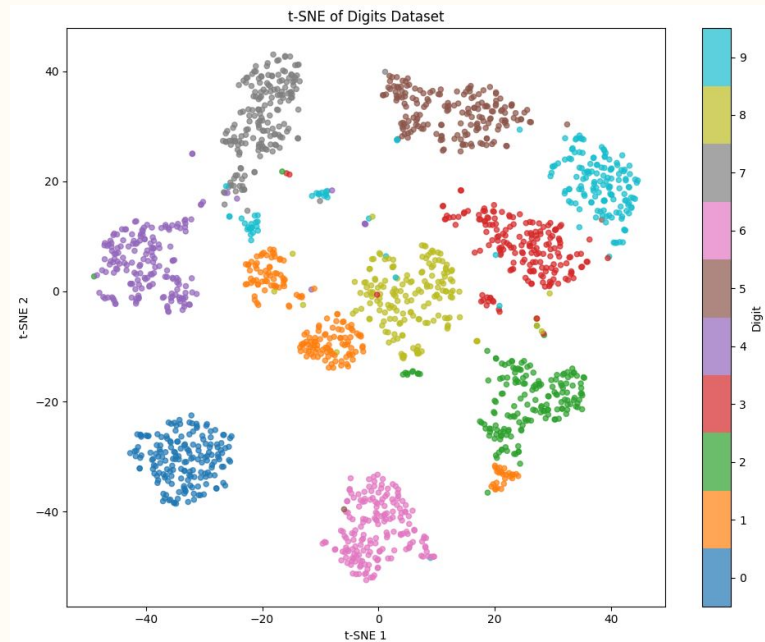
Otros famoso dataset: dígitos escritos a mano

- 1797 imágenes de 8×8 píxeles
- Cada píxel es una variable (valor gris)
- ¡**64** variables!



Espacio 64D

t-SNE
→



Espacio 2D

2. Reducción de dimensiones: UMAP

Uniform **M**anifold **A**pproximation and **P**rojection

McInnes et al. (2018)

- Una técnica de reducción de dimensiones moderna **no-lineal** que preserva mejor la estructura global que t-SNE
- Ideas centrales:
 - Los datos se embeben en un ***manifold*** de **pocas dimensiones**
 - Preserva (aprox) tanto **estructura local** como **global**
 - Construido a partir de un grafo de vecinos cercanos
- Sirve para:
 - Visualizar dataset grandes y de muchas dimensiones
 - Preservar cercanía entre clusters
 - Pre-procesamiento para otros métodos (e.g. clasificación)

2. Reducción de dimensiones: UMAP

Ojo: necesitamos instalar **umap-learn**

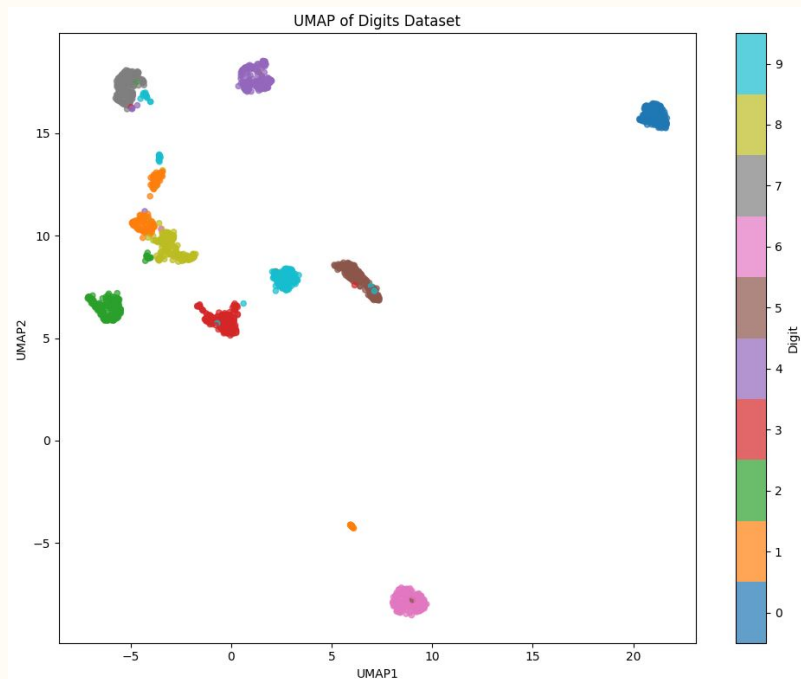
(sigue la misma API que scikit-learn, pero es una biblioteca independiente)

- Parámetros:
 - **n_neighbors**: estructura local (chico) vs global (grande)
 - **min_dist**: qué tan compactos son los clusters
- Fortalezas:
 - Suele ser **más rápido** que t-SNE
 - Funciona bien con **dataset grandes**
 - Comúnmente **mejor separación** de clusters
- Limitaciones:
 - No-lineal → difícil de **interpretar** los ejes nuevos
 - **Distancias** no preservadas exactamente
 - Sensible a elección de **parámetros**

2. Reducción de dimensiones: UMAP

Ejemplo: [umap.ipynb](#)

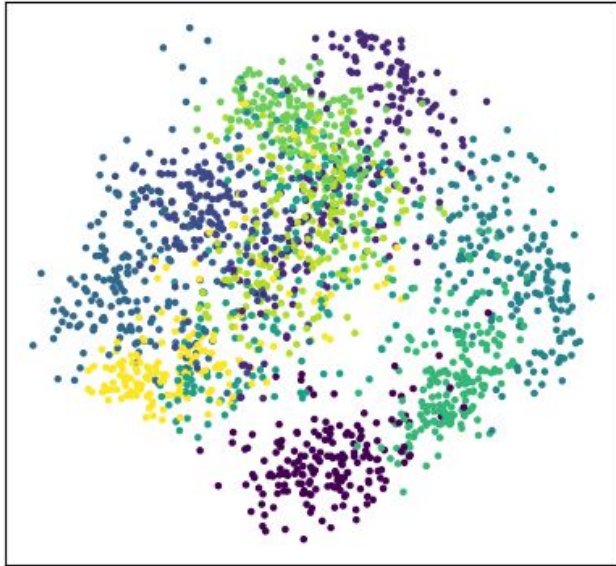
Dataset de dígitos escritos a mano



2. Reducción de dimensiones: comparación

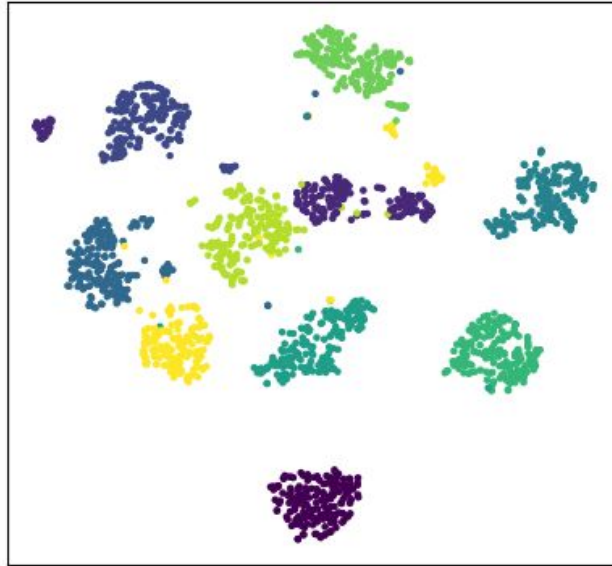
Comparación de resultados para el dataset de dígitos

PCA



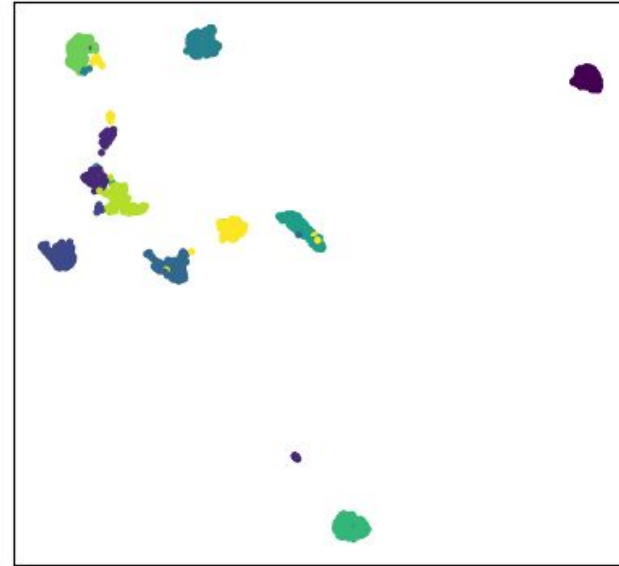
Preserva **varianza**

t-SNE



Preserva (aprox.)
estructura local

UMAP



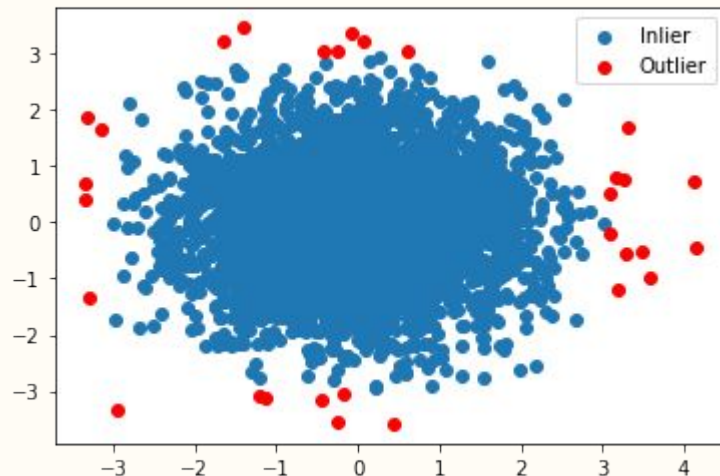
Preserva (aprox.)
estructura local y global

3. Detección de anomalías

- Objetivo: Identificar puntos que se **desvían fuertemente** de la mayoría de los datos

¿Qué es una anomalía?

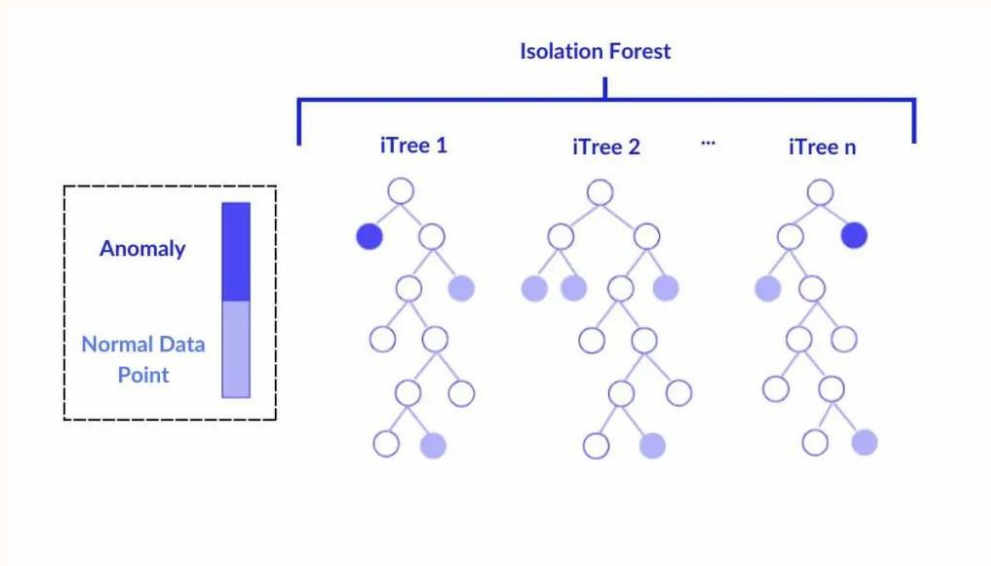
- Un evento **poco frecuente**
- Un **outlier** (no se conforma al patrón de la mayoría)
- Algo **no deseable** si se quieren construir modelos simples
- Un patrón **novedoso** o **inesperado** (¿reta ciencia actual?)



3. Detección de anomalías: Isolation Forest

Las **anomalías** son más fáciles de **aislar** que los datos normales justo porque están más “lejos” de la mayoría.

Estrategia: construir **árboles de decisión** (parecido a clustering jerárquico) al azar, y ver cuáles puntos quedan seguidos en **ramas aisladas**.



3. Detección de anomalías: Isolation Forest

Ejemplo: [isolation.ipynb](#)

